



# Programmierung und Deskriptive Statistik

BSc Psychologie WiSe 2025/26

Belinda Fleischmann und Dirk Ostwald

|          | Gruppe 1/2        | Gruppe 3          | Format         | Thema                                                |
|----------|-------------------|-------------------|----------------|------------------------------------------------------|
| 1        | Mi, 15.10.        | Do, 16.10.        | Seminar        | (1) Grundbegriffe der Informatik                     |
| 2        | Mi, 22.10.        | Do, 23.10.        | Seminar        | (2) Arithmetik und Variablen                         |
| 3        | Mi, 29.10.        | Do, 30.10.        | Übung          | (2) Arithmetik und Variablen                         |
| 4        | Mi, 05.11.        | Do, 06.11.        | Seminar        | (3) Vektoren und Matrizen                            |
| 5        | Mi, 12.11.        | Do, 13.11.        | Übung          | (3) Vektoren und Matrizen                            |
| <b>6</b> | <b>Mi, 19.11.</b> | <b>Do, 20.11.</b> | <b>Seminar</b> | <b>(4) Listen und Dataframes</b>                     |
| 7        | Mi, 26.11.        | Do, 27.11.        | Übung          | (4) Listen und Dataframes                            |
| 8        | Mi, 03.12.        | Do, 04.12.        | Seminar        | (5) Datenmanagement                                  |
| 9        | Mi, 10.12.        | Do, 11.12.        | Übung          | (5) Datenmanagement                                  |
|          | Mo, 15.12         |                   | Abgabe         | <i>Individuelleistung (1/2)</i>                      |
| 10       | Mi, 17.12.        | Do, 18.12.        | Seminar        | (6) Strukturiertes Progr.: Kontrollfluss, Debugging  |
| 11       | Mi, 07.01.        | Do, 08.01.        | Seminar        | (7) Häufigkeitsverteilungen                          |
| 12       | Mi, 14.01.        | Do, 15.01.        | Übung          | (7) Häufigkeitsverteilungen                          |
| 13       | Mi, 21.01.        | Do, 22.01.        | Seminar        | (8) Maße der zentralen Tendenz und Datenvariabilität |
| 14       | Mi, 28.01.        | Do, 29.01.        | Übung          | (8) Maße der zentralen Tendenz und Datenvariabilität |
|          | Mo, 02.02         |                   | Abgabe         | <i>Individuelleistung (2/2)</i>                      |

## (5) Listen und Dataframes

## Listen

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## **Listen**

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## **Listen**

### **Übersicht und Erzeugung**

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

### Übersicht und Erzeugung

Charakterisierung

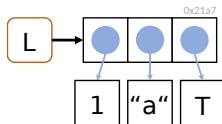
Indizierung

Arithmetik

Copy-on-modify

- Listen sind geordnete Folgen von R Objekten.
- Listen sind rekursiv, können also Objekte verschiedenen Datentyps enthalten.
- De facto enthalten Listen keine Objekte, sondern Referenzen zu Objekten.

```
L <- list(1,"a",T)
```



- Listen sind ein wesentlicher Baustein von Dataframes.

# Erzeugung

## Direkte Konkatenation von Listenelementen mit list()

```
L <- list(c(1, 4, 5),  
         matrix(1:8, nrow = 2),  
         exp)  
print(L)
```

# Liste mit einem Vektor,  
# einer Matrix und  
# einer Funktion

```
[[1]]  
[1] 1 4 5  
  
[[2]]  
  [,1] [,2] [,3] [,4]  
[1,]    1    3    5    7  
[2,]    2    4    6    8  
  
[[3]]  
function (x) .Primitive("exp")
```

## Listen können Elemente von Listen sein

```
L <- list(list(1))  
print(L)
```

# Liste mit Element 1 in einer Liste

```
[[1]]  
[[1]][[1]]  
[1] 1
```

## c() kann zum Verbinden von Listen genutzt werden

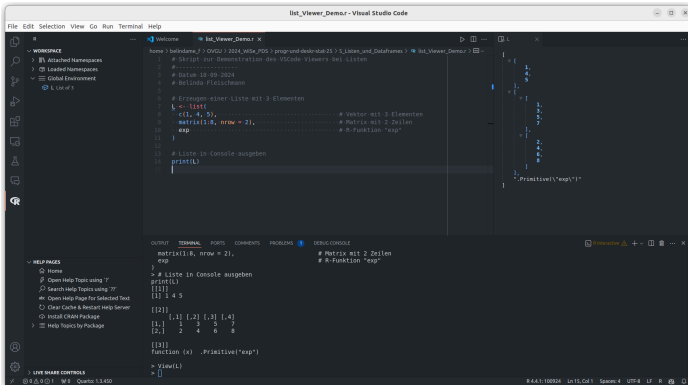
```
L <- c(list(pi), list("a"))  
print(L)
```

# Konkatenation zweier Listen

```
[[1]]  
[1] 3.141593  
  
[[2]]  
[1] "a"
```

# VSCode Interactive Viewers

## List Viewer



Mit dem Befehl `View()` oder im R **WORKSPACE** → **Global Environment** über das View Symbol



neben entsprechendem Objekt

[VS Code Wiki - Interactive viewers](#)

## **Listen**

Übersicht und Erzeugung

**Charakterisierung**

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

# Charakterisierung

Der Datentyp von Listen ist `list`

```
L <- list(1:2, "a", log)      # Erzeugung einer Liste  
typeof(L)                   # Typenbestimmung
```

```
[1] "list"
```

`length()` gibt die Anzahl der Toplevel Listenelemente aus

```
L <- list(1:2, list("a", pi), exp)  # Liste mit drei Toplevel-Elementen  
length(L)                          # length() ignoriert Elementinhalte, length() von L ist also 3
```

```
[1] 3
```

Die Dimension, Zeilen-, und Spaltenanzahl von Listen ist `NULL`

```
L <- list(1:2, "a", sin)          # eine Liste  
dim(L)                           # Die Dimension von Listen ist NULL
```

```
NULL
```

```
nrow(L)                          # Die Zeilenanzahl von Listen ist NULL
```

```
NULL
```

```
ncol(L)                          # Die Spaltenanzahl von Listen ist NULL
```

```
NULL
```

## **Listen**

Übersicht und Erzeugung

Charakterisierung

**Indizierung**

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Listenelement vs. Listenelementinhalt

Einfache eckige Klammern [ ] indizieren **Listenelemente als Listen**

```
L <- list(1:3, "a", exp)      # eine Liste  
l1 <- L[1]                  # Indizierung eines Listenelements  
print(l1)
```

```
[[1]]  
[1] 1 2 3
```

```
typeof(l1)                  # Typbestimmung von l1
```

```
[1] "list"
```

Doppelte eckige Klammern [[ ]] indizieren den **Inhalt von Listenelementen**

```
L <- list(1:3, "a", exp)      # eine Liste  
i2 <- L[[2]]                 # Indizierung des Listenelementinhalts  
print(i2)
```

```
[1] "a"
```

```
typeof(i2)                  # Typbestimmung von i2
```

```
[1] "character"
```

## Ersetzen von Listenelement(inhalt)

### Ersetzen eines Listenelements mit einfachen eckige Klammer [ ]

```
L      <- list(1:3, "a", exp)      # eine Liste
L[1]   <- 4:6                     # Ersetzung des 1. Listenelementes mit gewünschtem Inhalt
```

Warning in L[1] <- 4:6: number of items to replace is not a multiple of replacement length

```
print(L[1])                      # -> keine Typkonversion vector zu list; erzeugt stattdessen warning message
                                   # und übernimmt nur erstes Element des Vektors
```

```
[[1]]
[1] 4
```

```
L[1]   <- list(4:6)              # Ersetzung des 1. Listenelementes mit einer Liste des gewünschten Inhalts
print(L[1])
```

```
[[1]]
[1] 4 5 6
```

### Ersetzen des Listenelementinhalts über doppelte eckige Klammern [[ ]]

```
L[[1]] <- 7:9                    # Ersetzung des 1. Listenelementinhaltes mit gewünschtem Inhalt
L[[3]] <- "c"                    # Ersetzung des 3. Listenelementinhaltes mit gewünschtem Inhalt
print(L)
```

```
[[1]]
[1] 7 8 9
```

```
[[2]]
[1] "a"
```

```
[[3]]
[1] "c"
```

## Prinzipien der Listenindizierung

Die Prinzipien der Listenindizierung sind analog zur Vektorindizierung.

### Vektoren positiver Zahlen adressieren entsprechende Elemente

```
L <- list(1:3, "a", pi)           # eine Liste  
print(L[c(1, 3)])                # 1. und 3. Listenelement
```

```
[[1]]  
[1] 1 2 3
```

```
[[2]]  
[1] 3.141593
```

### Vektoren negativer Zahlen adressieren komplementäre Elemente

```
print(L[-c(1, 3)])               # 2. Listenelement
```

```
[[1]]  
[1] "a"
```

### Logische Vektoren adressieren Elemente mit TRUE.

```
print(L[c(TRUE, TRUE, FALSE)])   # 1. und 2. Listenelement
```

```
[[1]]  
[1] 1 2 3
```

```
[[2]]  
[1] "a"
```

# Indizierung

Zur nicht-numerischen Adressierung können Listenelementen Namen gegeben werden.

```
L <- list(marie = 1:3,           # eine Liste mit benannten Elementen
         lise  = "a",
         emmy  = exp)
print(L)
```

```
$marie
[1] 1 2 3
```

```
$lise
[1] "a"
```

```
$emmy
function (x) .Primitive("exp")
```

Namen werden automatisch als Attribut `names` gespeichert und können mit `attributes()` angezeigt werden

```
print(attributes(L))
```

```
$names
[1] "marie" "lise"  "emmy"
```

Listenelementen können auch mit `names()` Namen gegeben werden

```
K <- list(1:2, TRUE)           # eine unbenannte Liste
names(K) <- c("Frodo", "Sam")  # Namensgebung mit names()
print(K)
```

```
$Frodo
[1] 1 2
```

```
$Sam
[1] TRUE
```

Mit Namen können **Listenelemente** ([ ]) und **Listenelementinhalte** ([[ ]]) indiziert werden.

```
L <- list(marie = 1:3,      # eine Liste mit benannten Elementen
         lise  = "a",
         emmy  = exp)
L["emmy"]                # Listenelementindizierung
```

```
$emmy
function (x) .Primitive("exp")
L[["emmy"]]              # Listenelementinhaltsindizierung
```

```
function (x) .Primitive("exp")
```

Mit dem **\$**-Operator können **Listenelementinhalte** indiziert werden.

```
L <- list(marie = 1:3,      # eine Liste mit benannten Elementen
         lise  = "a",
         emmy  = exp)
L$marie                  # Listenelementinhalt
```

```
[1] 1 2 3
L$lise                   # Listenelementinhalt
```

```
[1] "a"
L$emmy                   # Listenelementinhalt
```

```
function (x) .Primitive("exp")
```

## **Listen**

Übersicht und Erzeugung

Charakterisierung

Indizierung

**Arithmetik**

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

Listenarithmetik ist nicht definiert, da Listenelemente unterschiedlichen Typs sein können

```
L1 <- list(1:3, "a")      # eine Liste
L2 <- list(TRUE, exp)     # eine Liste
L1+L2                    # Versuch der Listenaddition
```

Error in L1 + L2: non-numeric argument to binary operator

Listenelementinhalte können bei Passung jedoch arithmetisch verknüpft werden

```
L1 <- list(1:3, 1)       # eine Liste
L2 <- list(4:6, exp)      # eine Liste
L1[[1]] + L2[[1]]        # Addition der 1. Listenelementinhalte, [1+4, 2+5, 3+6]
```

```
[1] 5 7 9
```

```
L2[[2]](L1[[2]])        # Anwendung des 2. Listenelementinhalts auf den 1. Listenelementinhalt -> exp(1)
```

```
[1] 2.718282
```

## Listen

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

**Copy-on-modify**

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

# Copy-on-modify

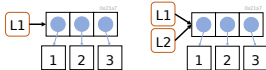
Wie bei Vektoren gilt bei Listen das Copy-on-Modify Prinzip.

“Shallow copy”: Listenobjekt wird kopiert, aber nicht die gebundenen Objekte.

`lobstr::ref()` erlaubt es, dieses Verhalten zu studieren.

`L1 = list(1,2,3)`

`L2 = L1`



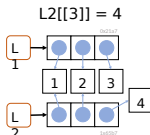
```
L1 <- list(1, 2, 3) # Erzeugen einer Liste als Objekt (z.B. 0x1a3)
L2 <- L1           # L2 wird das selbe listenobject wie L1 zugewiesen
lobstr::ref(L1, L2) # Ausgabe der Referenzen beider Listen
```

```
[1:0x652eab7fa498] <list>
[2:0x652eab9a01c0] <dbl>
[3:0x652eab9a0000] <dbl>
[4:0x652eab99fe40] <dbl>
```

```
[1:0x652eab7fa498]
```

# Copy-on-modify

## Änderung nur eines Listenelements



```
L2[[3]] <- 4          # Copy-on-Modify mit shallow Objekt Kopie
lobstr::ref(L1, L2)    # Ausgabe der Referenzen beider Listen
```

```
[1:0x652eab7fa498] <list>
[2:0x652eab9a01c0] <dbl>
[3:0x652eab9a0000] <dbl>
[4:0x652eab99fe40] <dbl>

[5:0x652eac5423b8] <list>
[2:0x652eab9a01c0]
[3:0x652eab9a0000]
[6:0x652eac4f1270] <dbl>
```

Anmerkung: Die Referenzen der Objekte beziehen sich auf die adressierten Speicherzellen im Arbeitsspeicher. Da bei jeder Neuerstellung von Objekten, neue Speicherzellen vergeben werden, variieren die genauen Adressen bei jeder Neuerstellung von Objekten. Entsprechend werden die mit `lobstr::ref()` ausgegebenen Referenzen nicht mit denen in der Abbildung oder bei Replikation zu Hause übereinstimmen.

## Listen

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Listen

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

**Übersicht und Erzeugung**

Charakterisierung

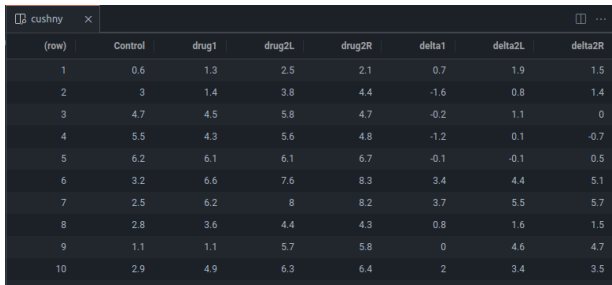
Indizierung

Arithmetik

Copy-on-modify

# Übersicht

- Dataframes sind die zentrale Datenstruktur in R.
- Dataframes stellt man sich am besten als Tabelle vor.
- Die Zeilen und Spalten der Tabelle haben Namen.



The image shows a screenshot of an R Dataframe named 'cushny'. The Dataframe has 10 rows and 8 columns. The columns are labeled (row), Control, drug1, drug2L, drug2R, delta1, delta2L, and delta2R. The rows are numbered 1 through 10. The values in the columns are as follows:

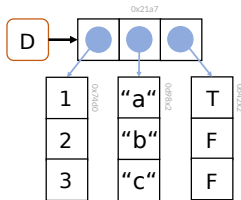
| (row) | Control | drug1 | drug2L | drug2R | delta1 | delta2L | delta2R |
|-------|---------|-------|--------|--------|--------|---------|---------|
| 1     | 0.6     | 1.3   | 2.5    | 2.1    | 0.7    | 1.9     | 1.5     |
| 2     | 3       | 1.4   | 3.8    | 4.4    | -1.6   | 0.8     | 1.4     |
| 3     | 4.7     | 4.5   | 5.8    | 4.7    | -0.2   | 1.1     | 0       |
| 4     | 5.5     | 4.3   | 5.6    | 4.8    | -1.2   | 0.1     | -0.7    |
| 5     | 6.2     | 6.1   | 6.1    | 6.7    | -0.1   | -0.1    | 0.5     |
| 6     | 3.2     | 6.6   | 7.6    | 8.3    | 3.4    | 4.4     | 5.1     |
| 7     | 2.5     | 6.2   | 8      | 8.2    | 3.7    | 5.5     | 5.7     |
| 8     | 2.8     | 3.6   | 4.4    | 4.3    | 0.8    | 1.6     | 1.5     |
| 9     | 1.1     | 1.1   | 5.7    | 5.8    | 0      | 4.6     | 4.7     |
| 10    | 2.9     | 4.9   | 6.3    | 6.4    | 2      | 3.4     | 3.5     |

Anmerkung: Der in diesem Beispiel verwendete Datensatz `cushny` ist im R Paket `psychTools` enthalten. Nach Installation und Laden des Pakets kann der Datensatz mit dem Befehl `'data(cushny)'` in den Workspace geladen werden. Mehr Details zu `cushny` und weiteren Datensätzen [hier](#).

# Übersicht

- Technisch ist ein Dataframe eine Liste, deren Elemente R Vektoren gleicher Länge sind.
- Die Listenelemente entsprechen den Spalten einer Tabelle.
- Die Vektorelemente gleicher Position entsprechen den Zeilen einer Tabelle.

```
D <- data.frame(c(1,2,3),  
                c("a", "b", "c"),  
                c(T,F,F))
```



# Erzeugung

## `data.frame()` erzeugt einen Dataframe

```
D <- data.frame(x = letters[1:4], # 1. Spalte mit Name x
               y = 1:4,          # 2. Spalte mit Name y
               z = c(T, T, F, T)) # 3. Spalte mit Name z
print(D)
```

```
  x y    z
1 a 1 TRUE
2 b 2 TRUE
3 c 3 FALSE
4 d 4 TRUE
```

Die Spalten eines Dataframes können offenbar unterschiedlichen Typs sein.

⇒ rekursive Datenstruktur.

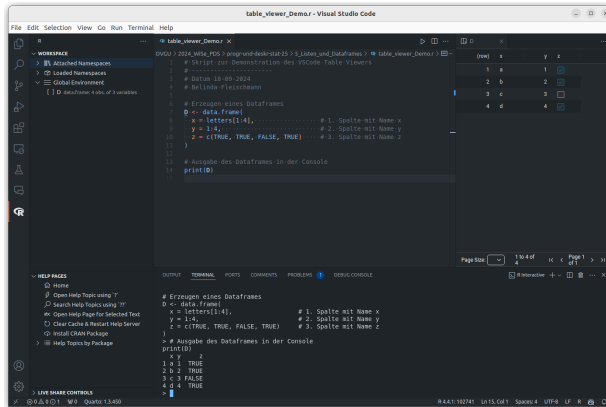
Allerdings müssen alle Spalten die gleiche Länge haben.

```
D <- data.frame(x = letters[1:4], # 1. Spalte mit Länge 4
               y = 1:4,          # 2. Spalte mit Länge 4
               z = c(T, T, F))   # 3. Spalte mit Länge 3
```

```
Error in data.frame(x = letters[1:4], y = 1:4, z = c(T, T, F)): arguments imply differing number of rows: 4, 3
```

# VSCode Interactive Viewers

## Table Viewer



Mit dem Befehl `View()` oder im R **WORKSPACE** → **Global Environment** über das View Symbol



neben entsprechendem Objekt

[VS Code Wiki - Interactive viewers](#)

## Listen

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

**Charakterisierung**

Indizierung

Arithmetik

Copy-on-modify

# Charakterisierung

Ein Dataframe hat `names()`, `rownames()`, `colnames()`

```
D <- data.frame(age   = c(30, 35, 40, 45), # 1. Spalte
               height = c(178, 189, 165, 171), # 2. Spalte
               weight  = c(67, 76, 81, 92)) # 3. Spalte
names(D)                                     # names gibt die Spaltennamen aus
```

```
[1] "age" "height" "weight"
```

```
colnames(D)                                     # colnames entspricht names
```

```
[1] "age" "height" "weight"
```

```
rownames(D)                                     # default rownames sind 1,2,...
```

```
[1] "1" "2" "3" "4"
```

Ein Dataframe `nrow()` Zeilen und `length()` bzw. `ncol()` Spalten

```
nrow(D)                                     # Zeilenanzahl
```

```
[1] 4
```

```
ncol(D)                                     # Spaltenanzahl
```

```
[1] 3
```

```
length(D)                                     # Länge ist die Spaltenanzahl
```

```
[1] 3
```

# Charakterisierung

`View()` öffnet den Data Viewer.

```
View(D)
```

|  D | ×   |        |        | ... |
|-------------------------------------------------------------------------------------|-----|--------|--------|-----|
| (row)                                                                               | age | height | weight |     |
| 1                                                                                   | 30  | 178    | 67     |     |
| 2                                                                                   | 35  | 189    | 76     |     |
| 3                                                                                   | 40  | 165    | 81     |     |
| 4                                                                                   | 45  | 171    | 92     |     |

`str()` zeigt in kompakter Form wesentliche Aspekte eines Dataframes an.

```
str(D)
```

```
'data.frame':  4 obs. of  3 variables:
 $ age  : num  30 35 40 45
 $ height: num  178 189 165 171
 $ weight: num  67 76 81 92
```

Allgemein zeigt `str()` in kompakter Form wesentliche Aspekte eines R Objektes an.

# Attribute

Technisch sind Dataframes Listen mit Attributen für `names` und `row.names`.

`names` bezieht sich dabei auf die Spaltenbezeichnungen.

```
attributes(D)
```

```
$names
```

```
[1] "age"      "height" "weight"
```

```
$class
```

```
[1] "data.frame"
```

```
$row.names
```

```
[1] 1 2 3 4
```

Dataframes gehören formal zur Datenstruktur "list" und sind von class "data.frame".

```
typeof(D)
```

```
[1] "list"
```

```
class(D)
```

```
[1] "data.frame"
```

## Listen

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

**Indizierung**

Arithmetik

Copy-on-modify

# Indizierung

Dataframes können sowohl wie Listen als auch wie Matrizen indiziert werden. Die Prinzipien der Indizierung entsprechen dabei denen von Vektoren.

Bei **einem** Index verhalten sich Dataframes wie **Listen**

```
D <- data.frame(x = letters[1:4], # 1. Spalte mit Name x
               y = 1:4,          # 2. Spalte mit Name y
               z = c(T, T, F, T)) # 3. Spalte mit Name z
print(D)
```

```
  x y    z
1 a 1 TRUE
2 b 2 TRUE
3 c 3 FALSE
4 d 4 TRUE
```

```
class(D) # D ist ein Dataframe
```

```
[1] "data.frame"
```

```
v <- D[1] # 1. Listenelement als Dataframe
v
```

```
 x
1 a
2 b
3 c
4 d
```

```
class(v) # v ist ein Dataframe
```

```
[1] "data.frame"
```

# Indizierung

Wie bei Listen können Dataframes mit doppelten Klammern indiziert werden, wodurch die Inhalte eines Listenelements adressiert werden:

```
w <- D[[1]]          # Inhalt des 1. Listenelements  
w
```

```
[1] "a" "b" "c" "d"
```

```
class(w)             # w ist ein character vector
```

```
[1] "character"
```

Die typische Indizierung eines Dataframes erfolgt mit der von den Listen bekannten \$ Indizierung, die den Inhalt der entsprechenden Dataframe Spalte adressiert:

```
y <- D$y             # $ zur Indizierung der y Spalte  
y
```

```
[1] 1 2 3 4
```

```
class(y)             # y ist ein Vektor vom Typ "integer"
```

```
[1] "integer"
```

## Bei **zwei** Indices verhalten sich Dataframes wie **Matrizen**

```
D <- data.frame(x = letters[1:4], # 1. Spalte mit Name x
               y = 1:4,          # 2. Spalte mit Name y
               z = c(T, T, F, T)) # 3. Spalte mit Name z
D[2:3,-2]                      # Zeilen 2-3 und alle Spalten außer 2
```

```
  x      z
2 b  TRUE
3 c FALSE
```

```
D[c(T, F, T, F),]           # 1. und 3. Zeile und alle Spalten
```

```
  x y      z
1 a 1  TRUE
3 c 3 FALSE
```

```
D[,c("x", "z")]             # Alle Zeilen und nur Spalten "x" und "z"
```

```
  x      z
1 a  TRUE
2 b  TRUE
3 c FALSE
4 d  TRUE
```

## Listen

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

**Arithmetik**

Copy-on-modify

# Arithmetik

Anders als bei Listen sind arithmetische Operationen für Dataframes definiert, setzen jedoch voraus, dass alle beteiligten Spalten numerisch und von gleicher Länge sind.

```
D1 <- data.frame(x = 1:4,      # 1. Spalte mit numerischem vector
                 y = 2:5)      # 2. Spalte mit numerischem vector
D2 <- data.frame(u = rep(TRUE, 4), # 1. Spalte mit logischem vector
                 v = rep("a", 4)) # 2. Spalte mit character vector

D1 + D2                                # Fehler bei nicht-numerischen Spalten
```

Error in FUN(left, right): non-numeric argument to binary operator

```
D3 <- data.frame(u = rep(1, 3), # 1. Spalte mit numerischem vector
                 v = rep(1, 3)) # 2. Spalte mit numerischem vector

D1 + D3                                # Dataframe-Addition bei numerischen Spalten
```

Error in Ops.data.frame(D1, D3): '+' only defined for equally-sized data frames

```
D4 <- data.frame(u = rep(1, 4), # 1. Spalte mit numerischem vector
                 v = rep(1, 4)) # 2. Spalte mit numerischem vector

D1 + D4                                # Dataframe-Addition bei numerischen Spalten
```

```
  x y
1 2 3
2 3 4
3 4 5
4 5 6
```

Arithmetische Operationen können auch auf einzelne Spalten angewendet werden, unabhängig davon, ob diese aus demselben oder aus verschiedenen Dataframes stammen.

```
D1[1] + D1[2] # Addition zweier Spalten (numerische Indizierung)
```

```
x
1 3
2 5
3 7
4 9
```

```
D1$x + D1$y # Addition zweier Spalten (Namensadressierung)
```

```
[1] 3 5 7 9
```

```
D1$x + D4$u # Spalten aus verschiedenen Dataframes
```

```
[1] 2 3 4 5
```

## Listen

Übersicht und Erzeugung

Charakterisierung

Indizierung

Arithmetik

Copy-on-modify

## Dataframes

Übersicht und Erzeugung

Charakterisierung

Indizierung

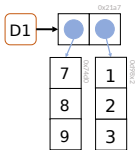
Arithmetik

**Copy-on-modify**

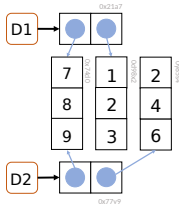
# Copy-on-modify

- Die Copy-on-Modify-Prinzipien, die für Listen gelten, gelten auch für Dataframes.
- Modifikation einer Spalte führt zur Kopie der entsprechenden Spalte
- Modifikation einer Zeile führt zur Kopie des gesamten Dataframes

```
D1 <- data.frame(  
  c(7,8,9),c(1,2,3))
```



```
D2 <- D1  
D2[,2] <- D2[,2] * 2
```



```
D3 <- D1  
D3[2,] <- D3[2,] * 3
```

